

Lecture 12

Software testing tools

Outline

- Automated testing
- Selenium
- Apache JMeter

References

- Daniel Galin, Software Quality Assurance: From theory to implementation, chapter 10
- Boni Garcia, Hands-On Selenium WebDriver with Java
- Antonio Gomes Rodrigues, Mastering Apache JMeter

References

- Daniel Galin, Software Quality Assurance: From theory to implementation, chapter 10
- Boni Garcia, Hands-On Selenium WebDriver with Java
- Antonio Gomes Rodrigues, Mastering Apache JMeter

Automated testing

Automated testing

- **The Concept:** Integrating computerized tools into the software testing process
- **Core Goals:**
 - Save time, cut costs
 - Improve accuracy
 - Generate better statistical reports
- **The Process:** Involves test planning, design, case preparation, computerized performance, reporting, and regression testing

Types of Automated test

- **Code Auditing:** Checks code compliance against specified coding standards
- **Coverage Monitoring (White-box):** Reports the percentage of code lines executed during tests
- **Functional Tests (Black-box):** Uses output comparators to identify errors and perform regression testing
- **Load Tests:** Simulates maximal user environments to measure reaction and processing times
- **Test Management:** Computerized tracking of test plans, execution, and error-correction follow-ups

Advantages and Disadvantages

Advantages

- Accuracy and completeness of performance
- Accuracy of results log and summary reports
- Comprehensive information
- Few manpower resources for test execution
- Shorter testing periods
- Performance of complete regression tests
- Performance of test classes beyond the scope of manual testing

Advantages and Disadvantages

Disadvantages

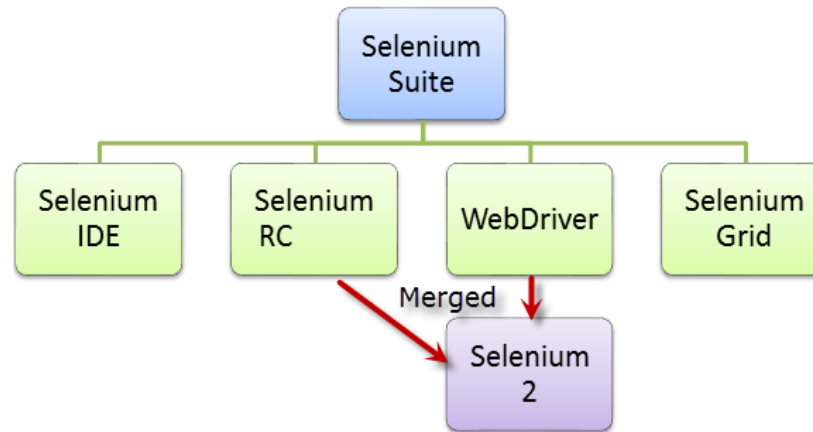
- High investments required in package purchasing and training
- High package development investment costs
- High manpower resources for test preparation
- Considerable testing areas left uncovered

Selenium

What is Selenium?

- **Core Definition:** An open-source test automation suite designed specifically for web applications
- **Key Capabilities:** Simulates real user interactions (clicking, typing, navigating) across different browsers and platforms
- **Flexibility:** Supports multiple programming languages (Java, C#, Python, Ruby, PHP, Perl, JavaScript, Kotlin)
- **Cost:** 100% free with a massive global support community

Selenium Suite



Consists of 4 main components tailored to specific testing needs

- **Selenium IDE:** Simple record-and-playback tool
- **Selenium RC (Remote control) & WebDriver:** The core automation engines (Merged into **Selenium 2**)
- **Selenium Grid:** Hub for parallel test execution across multiple machines

WebDriver vs. IDE

- **Selenium IDE (For Quick Prototypes):**
 - No support for complex logic (loops, if/else)
 - Hard to maintain as UI changes
 - Cannot be integrated into CI/CD pipelines
- **Selenium WebDriver (Industry Standard):**
 - **Full Control:** Write in Java, Python, or C# to handle complex business logic
 - **High Performance:** Fast, direct communication with the browser
 - **Scalable:** Seamlessly integrates with JUnit, external data, and CI/CD (Jenkins)

WebDriver Architecture

1. **Language Bindings:** Your test script (Java/Python code)
2. **W3C WebDriver Protocol:** Sends commands over the network formatted as JSON
3. **Browser Drivers:** Intermediary drivers (ChromeDriver, GeckoDriver, EdgeDriver) receive these commands and control the respective browser
4. **Real Browsers:** The browser executes the physical actions (clicking, typing) and returns the results

Locators - Finding Web Elements

- To interact with a page, WebDriver must pinpoint the exact HTML element in the DOM
- Common Locators:
 - `By.id()`: Fastest and most reliable
 - `By.name()`: Good alternative if ID is missing
 - `By.className()`: Locates elements by their CSS class
 - `By.tagName()`: Finds by HTML tag name
 - `By.linkText()`: Finds an `<a>` tag by its exact visible text
 - `By.partialLinkText()`: Finds an `<a>` tag by a portion of its visible text
 - `By.cssSelector()`: Highly flexible and fast execution
 - `By.xpath()`: Extremely powerful for complex DOM tree, but generally the slowest option

Browser & Navigation Commands

Category	Command	Description
Browser	<code>driver.get("url")</code>	Opens a static webpage (Waits for the page to fully load)
	<code>driver.getTitle()</code>	Retrieves the title of the current active tab
	<code>driver.getCurrentUrl()</code>	Retrieves the actual URL currently displayed (useful for testing redirects)
	<code>driver.close()</code>	Closes the currently active browser tab
	<code>driver.quit()</code>	Closes the entire browser and destroys the session (Best practice at the end of a test)
Navigation	<code>driver.navigate().to("url")</code>	Opens a webpage (Similar to <code>get()</code> but retains browser history)
	<code>driver.navigate().back()</code>	Simulates clicking the "Back" button in the browser
	<code>driver.navigate().refresh()</code>	Reloads the current page

Web Element Interaction Commands

Category	Command	Description
Action	<code>element.click()</code>	Clicks on the element (Buttons, links, checkboxes).
	<code>element.sendKeys("text")</code>	Types text into an input field.
	<code>element.clear()</code>	Clears any existing text from an input field.
	<code>element.submit()</code>	Simulates pressing the Enter key within a form.
Validation	<code>element.getText()</code>	Retrieves the visible text rendered on the screen for that element.
	<code>element.getAttribute("href")</code>	Gets the value of an HTML attribute (e.g., extracting a link URL).
State	<code>element.isDisplayed()</code>	Checks if the element is currently visible on the screen (Returns True/False).
	<code>element.isEnabled()</code>	Checks if the element is enabled and ready for interaction.
	<code>element.isSelected()</code>	Checks if a Checkbox or Radio button is currently ticked.

Handling Dropdowns

- Standard click() or sendKeys() commands are ineffective for standard HTML <select> dropdown menus
- Selenium provides a dedicated Select class to wrap the element and unlock built-in methods specifically designed
- **Implementation:**
 - **Locate** the <select> web element in the DOM
 - **Instantiate** a new Select object
 - **Execute** the selection using one of 3 strategies: selectByIndex(), selectByValue(), or selectByVisibleText()

Example - Handling Dropdowns

```
import org.openqa.selenium.support.ui.Select;

...

WebElement dropdown = driver.findElement(By.id("countryDropdown"));

Select countrySelect = new Select(dropdown);

countrySelect.selectByVisibleText("Vietnam");

// alternative strategies:
// countrySelect.selectByValue("VN");
// countrySelect.selectByIndex(1);

...
```

Integrating Assertions with JUnit 5

- Selenium is strictly a browser automation tool; it **does not** have built-in capabilities to determine if a Test Case Passes or Fails
- It must be paired with **JUnit 5** Assertion methods:
 - assertEquals: Useful for verifying page titles, text values, or exact URLs
 - assertTrue: Useful for checking if an element is visible
 - assertFalse: Verify that an element (like a loading spinner) is no longer present
 - assertNotNull: Verify that a web element was successfully located in the DOM

Synchronization

- Modern web pages load dynamically. If Selenium tries to interact before an element exists, it throws a `NoSuchElementException`
- Implicit Wait (Global Rule):
 - Set once per session
 - Tells WebDriver to wait up to X seconds for *any* missing element before failing
 - Syntax:

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

Synchronization

- Explicit Wait (The Smart Approach):
 - Highly targeted. It is the **industry best practice**
 - Pauses execution until a *specific condition* is met for a *specific element* (e.g., wait until a “Login” button is clickable)
 - Saves time: Stops waiting the exact moment the condition becomes true
 - Syntax:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));  
wait.until(ExpectedConditions.elementToBeClickable(By.id("loginBtn")));
```

Example - Basic Login Test

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;
import java.time.Duration;

class LoginTest {
    @Test
    public void testLogin() {
        WebDriver driver = new ChromeDriver();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        try {
            driver.get("https://example.com/login");
            driver.findElement(By.id("username")).sendKeys("testuser");
            driver.findElement(By.id("password")).sendKeys("pass123");
            driver.findElement(By.id("loginBtn")).click();
            assertEquals("https://example.com/dashboard",
driver.getCurrentUrl(), "Login failed!");
        } finally {
            driver.quit();
        }
    }
}
```

Apache JMeter

What is Apache JMeter?

- A Java open-source application designed to load test functional behavior and measure system performance
- **Primary Uses:**
 - **Performance Testing:** Analyzing overall system performance under different load types
 - **Load Testing:** Simulating expected user traffic to ensure the system can handle it
 - **Stress Testing:** Pushing the system beyond normal loads to find its breaking point

Why use JMeter for SQA?

- **Multi-Protocol Support:** Tests web applications (HTTP/HTTPS), databases (JDBC), APIs (REST/SOAP), and more
- **User Simulation:** Accurately mimics multiple concurrent users interacting with a system
- **Browser Behavior Emulation:** Can manage cookies, cache, and headers just like a real web browser to provide accurate test scenarios
- **Record and Playback:** Supports recording HTTP traffic via proxies or browser extensions (like BlazeMeter) to generate test scripts automatically

Anatomy of a JMeter Test Plan

- **A Test Plan:** is the root node containing all elements required to execute a performance test
- **Thread Group (The Users):**
 - The starting point of any test plan
 - Represents a pool of virtual users
 - Controls user behavior: **Number of Threads** (users), **Ramp-Up Period**, and **Loop Count**
- **Samplers (The Actions):**
 - Tell JMeter to send requests to a server
 - *Example:* **HTTP Request** sampler (e.g., simulating a user searching for a flight)

Anatomy of a JMeter Test Plan

- **Timers (The "Think Time"):**
 - By default, JMeter sends requests immediately
 - Timers add necessary delays to simulate real human interaction (e.g., a user reading a page before clicking)
- **Listeners (The Results):**
 - Gather and display the results of your test execution
 - *Examples:* **View Results Tree** (detailed request/response logs for debugging), **View Results in Table** (high-level execution metrics)

Handling Web Forms & Browser States

- **Passing Parameters:**
 - Real-world tests require sending data (e.g., selecting flight destinations)
 - HTTP Request samplers allow adding parameters (Name-Value pairs like fromPort=Boston and toPort=London) to simulate user inputs
- **Configuration Elements:**
 - HTTP Cookie Manager: Stores and sends cookies just like a web browser (crucial for maintaining user sessions/logins)
 - HTTP Cache Manager: Simulates browser caching to prevent downloading the same static assets (images, CSS) repeatedly, making load times more realistic

Script Creation Strategies

- **Manual Creation:**
 - Adding Thread Groups, Samplers, and setting up URLs/Paths manually.
 - Good for simple API endpoints or static pages
- **Recording (The Industry Standard):**
 - Uses a proxy or tools like the **BlazeMeter Chrome Extension**
 - **Process:** Click through the web workflow naturally -> The tool records all HTTP/S traffic -> Export as a .jmx file -> Import into Jmeter
 - **Benefit:** Captures all hidden requests, dynamic parameters, and headers automatically, saving massive amounts of scripting time

Summary

- Automated testing
- Selenium
- Jmeter